Desarrollo FullStack III (QUINTO SEMESTRE)

Tarea Investigación (Semana 2):

Herramientas y Estrategias

Nombre: Constanza Orellana

Contenedores

1. ¿Qué es Docker y qué problema resuelve en el desarrollo de software? Defina y relacione los conceptos de imagen, contenedor y Docker Hub, explicando cómo interactúan entre sí dentro del ciclo de vida de una aplicación dockerizada.

2. Mediante un diagrama comparativo, ilustre el despliegue de una misma aplicación web en dos escenarios: sin Docker (instalación directa en el servidor) y con Docker (usando contenedores). Identifique en cada caso los problemas o ventajas que presenta cada enfoque.

3. Explique cómo Docker se integra en una arquitectura de microservicios. Usando como ejemplo una aplicación de su elección con al menos tres servicios, elabore un diagrama que muestre cómo cada microservicio se empaqueta en su propio contenedor, indicando su imagen, puerto y base de datos independiente.

4. Describa el proceso completo a través de un diagrama para 6, identificando las herramientas necesarias en cada etapa. Elabore un diagrama de flujo paso a paso que muestre desde la escritura del Dockerfile hasta la ejecución del contenedor, indicando en qué herramienta se realiza cada acción y qué comando se ejecuta.

Kubernetes

1. ¿Qué es Kubernetes, cuál es su propósito dentro de una arquitectura de software moderna y por qué se le conoce como un "orquestador de contenedores"?

2. ¿Cuáles son los componentes principales de la arquitectura de Kubernetes y qué función cumple cada uno dentro del clúster? Ejemplifique con un diagrama.

3. ¿En qué se diferencia Kubernetes de Docker? Ejemplifique con un diagrama.

4. ¿Qué ventajas ofrece Kubernetes frente a un despliegue tradicional de aplicaciones en servidores físicos o virtuales, y en qué tipos de proyectos su uso está justificado? Ejemplifique.

CONTENEDORES

1.- ¿Qué es Docker?

En el desarrollo de software actual, la portabilidad y el despliegue eficiente son fundamentales. Las diferencias entre entornos suelen generar problemas de compatibilidad, y para resolverlos surge Docker, una tecnología de virtualización ligera que ayuda a empaquetar aplicaciones y dependencias en contenedores, asegurando consistencia y la facilidad de ejecución en cualquier sistema.

Docker es una plataforma de software que nos permite **crear, desplegar y ejecutar** aplicaciones dentro de **contenedores**.

- **Imagen:** archivo inmutable que contiene el sistema base, librerías y la aplicación.
- **Contenedor:** instancia en ejecución de una imagen, aislada y portable.
- **Docker Hub:** repositorio público/privado donde se almacenan y distribuyen imágenes.

Problemas que resuelve

Su principal aporte es resolver el problema de la falta de consistencia entre los entornos de desarrollo, pruebas y producción, evitando el error de “funciona en mi máquina”.

Como ejemplo tenemos el siguiente caso:

Un desarrollador crea una aplicación con Python 3.10 en su computadora, pero al desplegarla en el servidor que solo tiene Python 3.8, la aplicación falla. Con Docker, la aplicación y sus dependencias se empaquetan en un contenedor, asegurando que funcione igual en cualquier entorno sin importar las versiones instaladas.

Relación en el ciclo de vida:

El desarrollador escribe un **Dockerfile**, se construye una **imagen**, se publica en **Docker Hub**, y desde allí se descarga para crear y ejecutar **contenedores** en cualquier servidor.

2.-Despliegue de una misma aplicación web

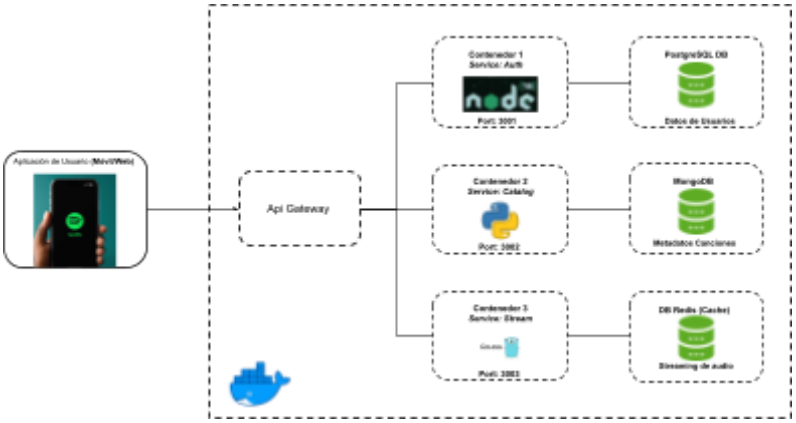
	Con Docker	Sin Docker (Tradicional)
Dependencias	Las dependencias **se empaquetan junto con la aplicación** en una imagen inmutable, garantizando que funcionen igual en cualquier entorno.	Se instalan en el sistema operativo host. **Pueden ocurrir conflictos de versiones** (como el "a mí me funciona en mi máquina").
Portabilidad	**Altísima.** Un contenedor funciona en cualquier lugar que tenga Docker instalado (laptop, servidor, nube).	**Depende** mucho de la **configuración específica** del sistema operativo, librerías y rutas del servidor destino.
Aislamiento	**Aislamiento a nivel** de procesos y sistema de archivos (namespaces). Los contenedores no ven los procesos de otros, aunque comparten el kernel.	Las aplicaciones comparten el mismo entorno y recursos, significa que **un fallo en una app puede afectar** a otras.

Despliegue	**Rápido y consistente**. Usando herramientas como docker-compose o Kubernetes, se despliega en segundos mediante imágenes preconfiguradas.	**Manual o mediante scripts** complejos de configuración (Ansible, Chef). Es más propenso a errores humanos.
Velocidad	**Arranque casi instantáneo** (milisegundos/segundos) ya que no inician un sistema operativo completo.	**Requiere iniciar la aplicación** y, a veces, dependencias del sistema, lo que hace el proceso más lento.
Escalabilidad	**Muy ágil.** Se pueden lanzar **múltiples instancias** (contenedores) de la aplicación rápidamente.	**Lenta.** Escalar implica configurar nuevos servidores o **instalar la app de nuevo en una nueva** máquina virtual.

3.-Integración de Docker con microservicios en una app

Esta arquitectura de Spotify utiliza Docker para desglosar la plataforma en tres microservicios autónomos (Auth, Catalog y Stream), cada uno empaquetado en su propio contenedor con un stack tecnológico específico (Node.js, Python y Go).

La integración se logra mediante un API Gateway que centraliza las peticiones de los usuarios y las distribuye a través de una red virtual de Docker hacia los contenedores correspondientes, los cuales operan de forma aislada en los puertos 3001, 3002 y 3003. Este diseño garantiza la persistencia e independencia de datos al asignar a cada servicio su propia base de datos (PostgreSQL, MongoDB y Redis), permitiendo que Spotify escale horizontalmente solo los componentes necesarios sin afectar la estabilidad global del sistema.



4.- Proceso para dockerizar una aplicación

El proceso de dockerización comienza en un editor como VS Code, donde se redacta el Dockerfile con las instrucciones del entorno. Mediante el Docker Engine y el comando docker build, este archivo se transforma en una imagen estática que contiene la aplicación y sus dependencias. Posteriormente, el flujo permite decidir si la imagen se almacena localmente o se sube a un registro como Docker Hub (docker push). Finalmente, el Docker Runtime utiliza el comando docker run para convertir esa

imagen en un contenedor vivo y funcional, cuya ejecución y puertos se monitorean con docker ps para asegurar que el servicio (como los de Spotify) esté operativo y accesible vía web.

KUBERNETES

1.- ¿Qué es Kubernetes?

Kubernetes es una plataforma de código abierto diseñada para automatizar la implementación, el escalado y la gestión de aplicaciones en contenedores.

Automatiza las tareas operativas de la administración de contenedores e incluye comandos integrados para implementar aplicaciones, actualizarlas, escalarlas a fin de que se ajusten a tus necesidades, supervisarlas y mucho más. Todo esto facilita la administración de las aplicaciones.

Su propósito dentro de una arquitectura de software moderna

Esta plataforma está diseñada para gestionar y escalar aplicaciones contenedorizadas en un clúster de máquinas. Automatiza la implementación, el escalado y las operaciones de los contenedores de aplicaciones.

¿Por qué se le conoce como un "orquestador de contenedores"?

Como se ha mencionado anteriormente, este automatiza la distribución y ejecución de contenedores en un clúster de máquinas. Actúa como un «director de orquesta», coordinando múltiples contenedores y nodos para garantizar que las aplicaciones funcionen correctamente, se autoreparen si fallan y escalen según la demanda.

2.- Componentes principales de la arquitectura de Kubernetes

La arquitectura de Kubernetes se divide en

Plano de Control (Control Plane / Master Nodes)	**kube-apiserver:** El frontend que expone la API de Kubernetes para interactuar con el clúster.
	etcd: Base de datos de clave-valor distribuida, almacena la configuración y estado del clúster.
	kube-scheduler: Decide en qué nodo se ejecutarán los nuevos pods basándose en la disponibilidad de recursos.
	kube-controller-manager: Ejecuta procesos controladores (nodos, replicación, endpoints) que mantienen el estado deseado.
	cloud-controller-manager: Interactúa con las API de proveedores de nube (AWS, Azure, GCP).
Nodos de Trabajo (Worker Nodes/Plano de Datos)	**kubelet:** Agente que asegura que los contenedores definidos en los pods funcionen correctamente en el nodo.
	kube-proxy: Gestiona las reglas de red en los nodos para permitir la comunicación de los servicios.
	Entorno de ejecución de contenedores (Container Runtime): Software responsable de la ejecución de contenedores (ej. containerd, CRI-O).

3.- Diferencia entre kubernetes y docker

La diferencia fundamental entre Docker y Kubernetes radica en su escala y propósito: Docker se encarga de empaquetar y ejecutar una aplicación en un contenedor individual, mientras que Kubernetes se encarga de coordinar y gestionar cientos o miles de esos contenedores a la vez.

Diferencias Clave

Característica	Docker	Kubernetes (K8s)
Unidad de medida	Se enfoca en el Contenedor.	Se enfoca en el **Pod** (que contiene uno o más contenedores).
Propósito	Crear, desplegar y ejecutar	Orquestar y escalar

	contenedores.	contenedores en un clúster de servidores.
Escalabilidad	Manual (o con Docker Swarm, más limitado).	Automática (Auto-scaling según la demanda).
Auto-curación	Si el contenedor muere, se queda abajo a menos que se reinicie manualmente.	Si un contenedor falla, K8s lo reinicia o lo reemplaza automáticamente.
Redes	Gestión de red básica entre contenedores.	Redes complejas que conectan múltiples nodos y servicios.

4.- ¿Qué ventajas ofrece Kubernetes frente a un despliegue tradicional de aplicaciones en servidores físicos o virtuales, y en qué tipos de proyectos su uso está justificado?

Kubernetes supera al despliegue tradicional porque automatiza la disponibilidad, la escalabilidad y la gestión de contenedores. A diferencia de los servidores físicos o virtuales, donde la administración de fallos y el balanceo de carga son manuales y rígidos, Kubernetes ofrece auto-reparación, orquestación inteligente y escalado dinámico. Esto lo convierte en la herramienta ideal para arquitecturas de microservicios y aplicaciones con tráfico variable que requieren actualizaciones constantes sin interrupciones.

Ejemplo:
En un e-commerce durante el Black Friday, Kubernetes detecta el aumento de tráfico y crea automáticamente réplicas adicionales del servicio de pagos. Al terminar el evento, reduce esas réplicas para optimizar los costos, evitando así la complejidad y lentitud de la configuración manual que exigiría un entorno tradicional.

Bibliografía

- Google Cloud. (2022). Kubernetes. Kubernetes. Rescatado en Marzo 22, 2026, desde https://cloud.google.com/learn/what-is-kubernetes?hl=es-419
- Stoychev, H. (2026, January 16). Kubernetes — The Orchestrator! - Telelink Business Services. tbs.tech. Rescatado en Marzo 22, 2026, desde https://www.tbs.tech/kubernetes-the-basics-core-concepts/
- The Kubernetes Authors. (2024). Kubernetes Documentation: What is Kubernetes? Rescatado en 23 de Marzo 2026, desde https://kubernetes.io/docs/concepts/overview/